

AdventureHub – Trekking Management Application

Modern Application Development I Project Report

Student Details:

Name: Deepesh Dey

Roll Number: 24F3002746

Email: 24f3002746@ds.study.iitm.ac.in

About Me: I am an Integrated M.Sc. Mathematics & Computing student at BIT Mesra and also pursuing the B.S. Degree in Data Science and Applications at IIT Madras. I enjoy building secure full-stack web applications and backend systems.

Description:

AdventureHub is a role-based trekking management web application developed using Flask. It enables administrators to manage treks, approve staff, assign treks, monitor bookings, and maintain users. Trek staff manage assigned treks while trekkers browse, book, and track trekking activities securely.

AI/LLM Declaration:

The extent of AI/LLM usage is around **10–15%**, limited to **code suggestions and documentation formatting**.

All final implementation logic, debugging, and integration were done manually.

Technologies Used:

Technology	Purpose
Python 3.13	Core language
Flask 3.1.3	Backend framework
Flask-Login	Authentication & sessions
SQLite (sqlite3)	Database
Jinja2	Template rendering
Bootstrap 5	Responsive UI
Werkzeug	Password hashing
OpenAPI YAML	API specification

DB Schema Design:

Tables:

- **Users** (id, name, email UNIQUE, password, role, status, created_at)
- **Treks** (id, trek_name, location, difficulty, description, start_date, end_date, slots, status, assigned_staff_id)
- **Bookings** (id, user_id, trek_id, booking_date, status, completed_date)

Design: Foreign keys maintain integrity, duplicate bookings are prevented, staff require admin approval, blacklisted users cannot log in, and passwords are securely hashed.

API Design:

JSON endpoints are documented in openapi.yaml. Resources include /api/treks, /api/bookings, /api/users and related endpoints. APIs reuse the authenticated Flask-Login session used by the web application.

Endpoint	Method	Description
/api/treks	GET	Retrieve all available treks (supports search/filtering)
/api/treks	POST	Create a new trek (Admin only)
/api/users	GET	Retrieve all registered users (Admin only)
/api/users/{userId}/status	PUT	Activate or blacklist a user (Admin only)
/api/bookings	GET	Retrieve booking records based on user role
/api/bookings	POST	Create a new trek booking (Trekker only)
/api/bookings/{bookingId}/cancel	PUT	Cancel an existing booking (Trekker only)

Architecture and Features:

- **app.py** – single Flask application file containing all routes, database table creation (raw SQLite via sqlite3, no ORM), and helper functions for business logic
- **templates/** – Jinja2 HTML templates, one dashboard per role (admin/staff/trekker) plus login/register/home, styled with Bootstrap 5 loaded via CDN
- **requirements.txt** – Python dependencies (Flask, Flask-Login, Werkzeug)
- **openapi.yaml** – API definition for the JSON endpoints under /api/...

The application follows a lightweight Flask architecture centered around app.py, which contains routing, business logic, authentication, database initialization and API endpoints. HTML templates are stored under templates/ and rendered using Jinja2.

Features include role-based dashboards (Admin, Trek Staff, Trekker), trek CRUD, staff approval, trek assignment, booking management, duplicate-booking prevention, booking history, profile management, responsive Bootstrap interface, HTML5 and server-side validation, secure session handling, progress charts, automatic database creation and documented REST APIs.

Folder Structure:

MAD_Pro_AdventureHub_24f3002746/ (root)

```
|— app.py
|— requirements.txt
|— README.md
|— openapi.yaml
|— templates/
|   |— home.html
|   |— login.html
|   |— register.html
|   |— admin_dashboard.html
|   |— staff_dashboard.html
|   |— user_dashboard.html
```

Demo Video | Google Drive Link:

<https://drive.google.com/file/d/1odddsn5qcsHpkb6UWkryoiAh1cJ9cI6t/view?usp=sharing>